

VLSI Design Internship – Task 4 Report

RTL Design of Finite State Machines (FSM) and Control Units

1. Introduction to Finite State Machines

Finite State Machines (FSMs) are mathematical models of computation used to design sequential logic circuits. An FSM consists of a finite number of states, transitions between those states, inputs, and outputs. FSMs are fundamental building blocks in modern digital design — they power control units in processors, communication protocols, embedded systems, and ASIC/FPGA controllers.

There are two main types of FSMs:

- **Moore Machine** – Output depends only on the current state.
- **Mealy Machine** – Output depends on both the current state and the current input.

An FSM consists of:

- States – distinct configurations the system can be in
- Inputs – external signals that trigger transitions
- Outputs – signals produced by the machine
- Transitions – rules for moving between states on a clock edge

2. Tools Used

- **Vivado (Xilinx)** – Writing, compiling, and simulating Verilog code
- **Built-in Waveform Viewer** – Analysing simulation output and state transitions
- **EDA Playground** – Online Verilog/SystemVerilog simulator for quick verification

3. Part B: Moore FSM Design (Basic State Machine)

A Moore FSM was designed that cycles through three states: $S0 \rightarrow S1 \rightarrow S2 \rightarrow S0$. In a Moore machine, the output is determined entirely by the current state, making it simpler to analyse and verify. The state is encoded as a 2-bit register.

Module Code:

```
module moore_fsm(
    input clk,
    input reset,
    output reg [1:0] state
);

parameter S0 = 2'b00, S1 = 2'b01, S2 = 2'b10;

always @(posedge clk or posedge reset) begin
    if (reset)
        state <= S0;
    else begin
        case(state)
            S0: state <= S1;
            S1: state <= S2;
            S2: state <= S0;
        endcase
    end
end
```

```

        S0: state <= S1;
        S1: state <= S2;
        S2: state <= S0;
        default: state <= S0;
    endcase
end
end

endmodule

Testbench:
module tb_moore_fsm;
    reg clk, reset;
    wire [1:0] state;

    moore_fsm uut (.clk(clk), .reset(reset), .state(state));

    always #5 clk = ~clk;

    initial begin
        clk = 0; reset = 1;
        #10 reset = 0;
        #200 $finish;
    end
endmodule

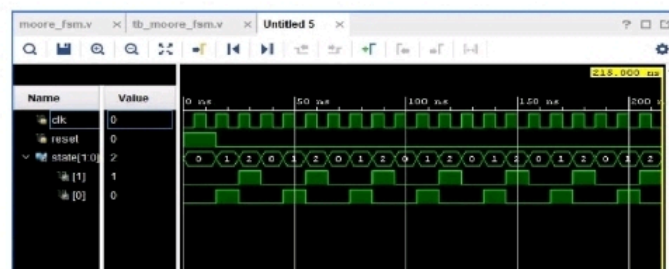
```

State Transition Table – Moore FSM:

Current State	Next State	State[1:0] Encoding
S0	S1	00 → 01
S1	S2	01 → 10
S2	S0	10 → 00

Simulation Waveform:

The waveform confirms that the state register cycles through 0 → 1 → 2 → 0 repeatedly on every positive clock edge. After reset is de-asserted, the FSM begins from S0 and transitions correctly through all three states.



4. Part C: Mealy FSM Design (Input-Based Output)

A Mealy FSM was designed where the output depends on both the current state and the current input. The output goes HIGH when input = 1 and the current state is S1. Because the output is combinationaly derived from the state and input, Mealy machines can respond one clock cycle faster than equivalent Moore machines.

Module Code:

```
module mealy_fsm(
    input clk,
    input reset,
    input in,
    output reg out
);

reg state;

always @(posedge clk or posedge reset) begin
    if (reset) begin
        state <= 0;
        out <= 0;
    end else begin
        case(state)
            0: begin
                if (in) begin
                    state <= 1;
                    out <= 0;
                end
            end
            1: begin
                if (in)
                    out <= 1;
                else begin
                    state <= 0;
                    out <= 0;
                end
            end
        endcase
    end
end

endmodule
```

Testbench:

```
module tb_mealy_fsm;
    reg clk, reset, in;
    wire out;

    mealy_fsm uut (.clk(clk), .reset(reset), .in(in), .out(out));

    always #5 clk = ~clk;

    initial begin
        clk = 0; reset = 1; in = 0;
        #10 reset = 0;
    end
endmodule
```

```

        #10 in = 1;
        #10 in = 0;
        #10 in = 1;
        #10 in = 1;
        #10 in = 0;
        #20 $finish;
    end
endmodule

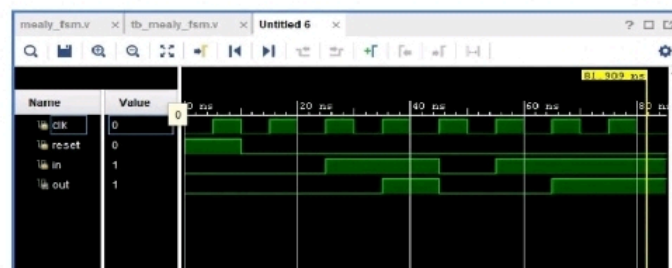
```

State Transition Table – Mealy FSM:

Current State	Input (in)	Next State	Output (out)
S0 (0)	0	S0 (0)	0
S0 (0)	1	S1 (1)	0
S1 (1)	0	S0 (0)	0
S1 (1)	1	S1 (1)	1

Simulation Waveform:

The waveform shows that the output goes HIGH only when the machine is in state S1 and the input is 1, confirming the Mealy behaviour. The output responds within the same clock cycle as the input change.



5. Part D: Traffic Light Controller (Industry-Level Mini Project)

A traffic light controller FSM was implemented as a practical, industry-level application. The controller cycles through Red, Green, and Yellow light states on every positive clock edge, simulating a real-world traffic signal system. The light output is a 3-bit register encoded using named parameters.

System Behaviour:

State	Light Output	light[2:0] Encoding
S0	Red	3'b100 (4)
S1	Yellow	3'b010 (2)
S2	Green	3'b001 (1)

Module Code:

```
module traffic_light(  
    input clk,  
    input reset,  
    output reg [2:0] light  
);  
  
parameter RED    = 3'b100,  
          YELLOW = 3'b010,  
          GREEN   = 3'b001;  
  
reg [1:0] state;  
  
always @(posedge clk or posedge reset) begin  
    if (reset) begin  
        state <= 0;  
        light <= RED;  
    end else begin  
        case(state)  
            0: begin state <= 1; light <= GREEN; end  
            1: begin state <= 2; light <= YELLOW; end  
            2: begin state <= 0; light <= RED;    end  
        endcase  
    end  
end  
  
endmodule
```

Testbench:

```
module tb_traffic_light;  
    reg clk, reset;  
    wire [2:0] light;  
  
    traffic_light uut (.clk(clk), .reset(reset), .light(light));  
  
    always #5 clk = ~clk;  
  
    initial begin  
        clk = 0; reset = 1;  
    end  
endmodule
```

```

        #10 reset = 0;
        #110 $finish;
    end
endmodule

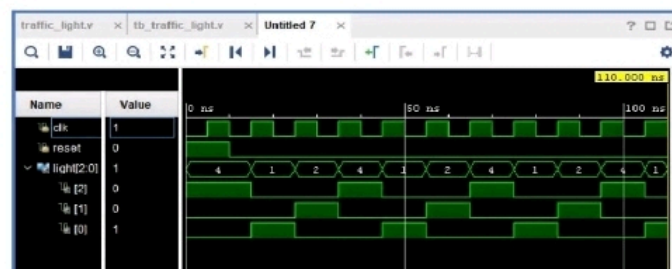
```

State Transition Table – Traffic Light:

Current State	Current Light	Next State	Next Light
S0 (Reset)	Red	S1	Green
S1	Green	S2	Yellow
S2	Yellow	S0	Red

Simulation Waveform:

The waveform confirms the light output cycling through values 4 (Red), 1 (Green), and 2 (Yellow) in the correct sequence on every clock edge. Reset initialises the controller to the Red state as expected.



6. Part E: Sequence Detector – Pattern Detection (1011)

A sequence detector FSM was designed to detect the binary pattern 1011 in a serial input stream. This is a classic FSM application used in communication systems and digital pattern recognition. The detector uses four states (S0–S3) representing how many bits of the target pattern have been matched so far. When the full sequence 1011 is detected, the output *detected* goes HIGH.

State Encoding:

State	Meaning	Bits Matched
S0 (0)	Initial / Reset	None
S1 (1)	Received '1'	'1'
S2 (2)	Received '10'	'10'
S3 (3)	Received '101' – waiting for last '1'	'101'

Module Code:

```

module sequence_detector(
    input clk,
    input reset,

```



```

        input in,
        output reg detected
    );

    reg [2:0] state;

    always @(posedge clk or posedge reset) begin
        if (reset) begin
            state <= 0;
            detected <= 0;
        end else begin
            detected <= 0;          // default: clear every cycle
            case(state)
                0: state <= (in) ? 1 : 0;
                1: state <= (in) ? 1 : 2;
                2: state <= (in) ? 3 : 0;
                3: begin
                    if (in) begin
                        detected <= 1;
                        state <= 1; // overlapping: '1' starts new attempt
                    end else
                        state <= 2; // partial match '10' still useful
                    end
                end
            endcase
        end
    end

endmodule

```

Testbench:

```

module tb_sequence_detector;
    reg clk, reset, in;
    wire detected;

    sequence_detector uut (
        .clk(clk), .reset(reset),
        .in(in), .detected(detected)
    );

    always #5 clk = ~clk;

    initial begin
        clk = 0; reset = 1; in = 0;
        #10 reset = 0;
        // Apply sequence: 1 0 1 1 (should detect at end)
        #10 in = 1;
        #10 in = 0;
        #10 in = 1;
        #10 in = 1; // detected should go HIGH here
        #10 in = 0;
        #10 in = 1;
        #10 in = 1;
        #10 in = 0;
        #10 in = 1;
        #10 in = 1; // second detection
        #20 $finish;
    end
endmodule

```

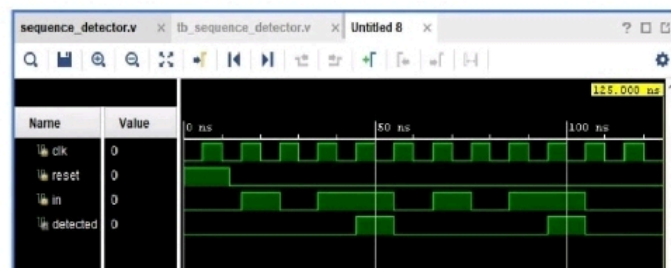
```
end
endmodule
```

State Transition Table – Sequence Detector (1011):

Current State	Input	Next State	Output (detected)
S0	0	S0	0
S0	1	S1	0
S1	0	S2	0
S1	1	S1	0
S2	0	S0	0
S2	1	S3	0
S3	0	S2	0
S3	1	S1	1

Simulation Waveform:

The waveform confirms that the *detected* output pulses HIGH exactly when the complete 1011 sequence is received at the input. The FSM correctly tracks partial matches through its intermediate states and handles overlapping sequences.



7. Testbench Development and Simulation Steps

All four FSM modules were verified using dedicated testbench files. Each testbench follows the same structure: a clock generator using `always #5 clk = ~clk` (10 ns clock period), an initial block to drive input stimulus, and observation of outputs in the waveform viewer.

General Simulation Flow:

- 1. Compile: `iverilog fsm.v tb_fsm.v -o a.out`
- 2. Run: `vvp a.out`
- 3. View: `gtkwave dump.vcd`
- 4. Verify correct state transitions, output timing, and reset behaviour

8. Observations

Moore FSM:

- The state register cycles through $S0 \rightarrow S1 \rightarrow S2 \rightarrow S0$ correctly on every positive clock edge. Reset immediately returns the FSM to $S0$. The output (state) changes only with the clock, confirming Moore behaviour.

Mealy FSM:

- The output responds to both the current state and the current input within the same clock cycle. Output goes HIGH only when state = $S1$ and input = 1, which is verified in the waveform. Mealy machines offer faster response than equivalent Moore machines.

Traffic Light Controller:

- The light output correctly cycles through Red (4) $\rightarrow$ Green (1) $\rightarrow$ Yellow (2) $\rightarrow$ Red (4). Reset initialises the system to the Red state. The 3-bit encoding using named parameters (RED, YELLOW, GREEN) improves code readability and maintainability.

Sequence Detector (1011):

- The detector correctly identifies the 1011 pattern and asserts the *detected* output for one clock cycle upon completion. The FSM handles overlapping sequences: after detecting 1011, a trailing '1' is treated as the beginning of a new match.

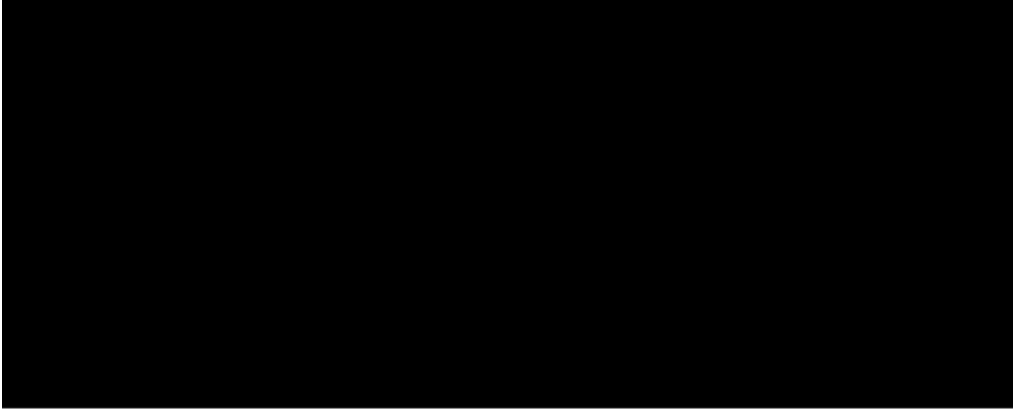
Common Observations:

- All FSMs use the `always @(posedge clk or posedge reset)` block, making them synchronous designs. Non-blocking assignments (`<=`) are used throughout, which is correct RTL practice for flip-flop-based sequential circuits. The **default** case in each case statement prevents synthesis of latches and ensures safe recovery from undefined states.

9. Conclusion

Task 4 successfully demonstrated the RTL design and simulation of four Finite State Machine circuits using Verilog HDL. The Moore FSM illustrated state-only output control, while the Mealy FSM showed how combining state and input produces faster output responses. The Traffic Light Controller provided a practical, real-world FSM application, and the Sequence Detector demonstrated pattern-recognition logic widely used in communication systems.





All designs were verified through simulation waveforms confirming correct state transitions, output timing, and reset behaviour. This task builds the foundation for designing complex control units in processors and ASIC/FPGA systems.

RTL DESIGN OF FINITE STATE MACHINES (FSM) AND CONTROL UNITS

